

# 智能在线编程练习平台项目

## 前端开发个人总结报告

报告人：黄嘉鑫 2023141461175

### 一、项目概述

本次参与开发的“智能在线编程练习平台”是基于 SpringBoot 后端框架搭配现代化前端技术栈搭建的一站式 Web 算法学习平台。项目主要为解决传统算法题库功能单一、评测机制固化、缺乏智能化指导等行业痛点，集成标准化题库管理、在线代码编辑、自动化程序评测、AI 代码智能分析、题目动态生成等核心功能。平台主要面向在校学生、编程爱好者、求职备考人员等群体，构建了“AI 出题→在线做题→自动判题→AI 评题→优化改进”的全流程智能化学习闭环。在本次项目中，我全程负责前端模块的整体设计与开发工作，顺利完成平台界面搭建、交互逻辑开发、前后端接口联调及用户体验优化等各项核心任务。

### 二、个人核心职责

作为项目前端开发负责人，我主要承担以下工作职责：

- 采用 Vue3 + TypeScript + Vite 技术栈，完成项目前端技术选型、工程架构搭建与落地配置工作；
- 负责平台核心页面 UI 搭建与交互逻辑开发，涵盖题库展示、在线代码编辑器、评测结果展示、AI 代码分析报告、学习数据可视化等核心业务模块；
- 对接后端各类业务接口，完成前端数据请求封装、数据解析处理与全局状态管理工作；
- 持续优化前端页面性能与用户使用体验，保障平台在不同设备、不同浏览器下的兼容性与稳定性；
- 配合项目团队完成功能联调与测试工作，及时排查修复前端各类 Bug，迭代优化平台功能。

### 三、主要工作成果

## （一）技术栈落地与架构搭建

结合项目业务需求与前端技术发展趋势，确定 Vue3 + TypeScript + Vite 为核心开发技术栈。自主完成 tsconfig.json、vite.config.ts 等核心工程化配置文件的编写与调试，搭建起模块化、高扩展性的前端项目架构。同时实现全局组件复用、路由分层管理、Pinia 状态管理等基础工程能力，统一项目开发规范，为后续整体业务功能的快速开发与迭代筑牢基础。

## （二）核心页面与功能开发

### 1. 题库管理模块

完成结构化题库展示页面的全流程开发，实现按知识点（数据结构、动态规划等）、难度等级（入门、简单、中等、困难）精准筛选、分类展示与关键词搜索功能。同时开发题目收藏、错题归集、历史练习记录回溯等交互功能，搭配前端本地缓存策略，有效保留用户操作数据，大幅提升平台整体使用体验。

### 2. 在线代码编辑模块

引入集成 Monaco Editor 专业代码编辑组件，实现 C/C++ 语言语法高亮、智能代码补全、代码自动缩进与格式化等专业编程功能。结合在线编程场景优化页面布局，合理规划题目描述区、代码编辑区、测试用例输入输出区，同时实现代码本地暂存、一键提交评测等核心交互逻辑，保障用户编程操作流畅稳定。

### 3. 评测与 AI 分析模块

独立开发自动化评测结果展示页面，精准展示答案正确（AC）、答案错误（WA）、编译错误（CE）、运行错误（RE）等各类评测状态，可视化呈现多组测试用例的对比详情。针对 AI 智能评题功能，完成算法复杂度分析、代码风格检测、代码可读性评估、代码缺陷诊断等分析报告的可视化展示，通过图表呈现、重点高亮标注等方式，让用户直观清晰地掌握代码问题与优化方向。

### 4. 学习数据可视化模块

基于 ECharts 可视化插件，搭建用户学习数据统计页面，实现刷题总量统计、正确率趋势分析、薄弱知识点梳理、做题耗时分布等数据图表展示效果。同时支持数据条件筛选、时间维度切换、分析报告导出等功能，满足用户日常学习复盘、能力总结的使用需求。

### 5. AI 题目生成模块

完成 AI 出题自定义表单页面开发，支持用户自主选择知识点、题目难度、题目类型等

参数。对接后端 AI 出题接口，实现题目实时生成、结果预览、在线编辑、入库保存及提交测评的完整业务流程，完善平台智能化出题能力。

### （三）接口联调与兼容性优化

全面对接后端 SpringBoot 业务接口，统一封装前端请求拦截器与响应拦截器，规范化处理接口异常、数据格式不统一等问题，保障前后端数据交互稳定。完成 Chrome、Firefox、Edge 等主流 PC 浏览器适配优化，调整页面响应式布局，适配不同分辨率设备展示需求。通过组件懒加载、路由缓存、图片资源压缩等多重性能优化手段，将核心页面首屏加载时间缩短至 2 秒以内，显著提升页面加载速度与使用流畅度。

### （四）测试与迭代优化

积极配合测试团队开展多轮前端功能测试，累计排查并修复界面适配错乱、交互逻辑异常、数据渲染错误等各类前端 Bug30 余个。结合团队内部评审意见及用户体验调研反馈，持续迭代优化代码编辑器操作流畅度、AI 分析报告展示形式、筛选功能交互逻辑等细节问题，不断提升平台的稳定性与易用性。

## 四、问题与解决思路

### （一）在线代码编辑器集成与适配问题

**问题描述：**Monaco Editor 组件整体体积较大，直接全局引入会导致页面首屏加载速度缓慢；同时该组件与 Vue3 响应式系统适配性不足，存在代码编辑光标错位、本地代码暂存失效等问题，严重影响用户编程体验。

**解决思路：**采用动态 import 异步加载方式拆分编辑器组件，实现组件按需加载，有效缩减首屏加载资源体积，提升页面加载速度。自定义封装适配指令，解决编辑器与 Vue3 响应式数据双向绑定异常问题，彻底修复光标错位 bug。同时结合 localStorage 本地存储技术，实时缓存用户编辑代码，杜绝页面刷新、意外退出导致的代码丢失问题，保障用户操作安全。

### （二）AI 分析报告可视化复杂度高的问题

**问题描述：**后端 AI 接口返回的代码分析数据维度繁杂，涵盖算法复杂度、代码风格、程序缺陷、可读性等多项内容，直接平铺展示会造成信息过载，用户无法快速抓取核心问题重点，阅读体验较差。

**解决思路：**梳理数据分析优先级，重构报告展示逻辑，将复杂度等级、核心代码缺陷等关键结论置顶展示，次要辅助信息采用折叠收纳方式，简化页面展示层级。引入标签化展示与颜色分级体系，用红色标注高危代码缺陷、黄色标注代码优化建议，提升信息辨识度。新增 AI 分析报告一键导出功能，满足用户深度复盘、存档学习的使用需

求。

### （三）接口联调数据一致性问题

**问题描述：**项目开发初期，后端部分接口返回数据格式、字段规范不统一，且多接口异步并发请求时，容易出现数据渲染错乱、数据覆盖等异常问题，影响页面正常展示。

**解决思路：**梳理所有接口返回数据规范，与后端开发团队沟通协商，统一全局数据返回格式与字段定义。采用 `Promise.allSettled` 方法管控异步并发请求，同时为所有接口请求增加加载中、加载完成、加载失败的状态标识，有效规避异步请求先后顺序混乱导致的数据渲染异常、数据覆盖等问题，保障页面数据展示的准确性与稳定性。

## 五、收获与成长

- 1. 技术能力显著提升：**通过本次项目实战，熟练掌握 `Vue3 + TypeScript + Vite` 整套前端工程化开发流程，精通 `Monaco Editor`、`ECharts` 等主流第三方组件的集成与二次开发，熟练掌握前端性能优化、复杂交互逻辑开发、前后端接口联调等核心技能，技术实操能力得到全方位夯实。
- 2. 团队协作能力增强：**全程与后端、测试、产品团队高效协同配合，熟悉项目整体开发流程与前后端联调核心逻辑，打破了只专注前端页面开发的单一思维，养成了从项目整体角度思考问题、推进开发的工作习惯。
- 3. 问题解决思维成型：**面对编辑器适配、大数据可视化、接口并发异常等复杂技术难题，逐步养成“拆解问题、定位核心症结、制定方案、落地验证”的标准化解决思路，独立排查问题、解决问题的能力大幅提升。
- 4. 产品思维逐步完善：**在功能开发过程中，始终立足用户使用场景优化交互细节，不再单纯实现基础功能，更加注重产品实用性与用户体验，深刻理解了“技术服务产品、产品服务用户”的开发核心逻辑。

## 六、不足与改进方向

- 1. 技术深度有待加强：**本次项目仅运用了懒加载、资源缓存等基础性能优化手段，对 `Web Worker`、虚拟列表、分片加载等高级前端优化技术的应用不足。后续将重点学习相关技术，并落地应用于题库海量题目渲染等场景，进一步提升项目性能。
- 2. 交互设计经验欠缺：**部分功能模块如 `AI` 题目生成表单的交互逻辑仍有优化空间，操作流程不够简洁流畅。后续将主动学习 `UED` 用户体验设计相关知识，结合用户使用反馈持续优化页面交互逻辑，提升产品易用性。
- 3. 设备兼容性覆盖不全：**项目仅完成主流新版浏览器的适配优化，对低版本浏览器、小众设备的兼容性测试不够全面。后续将完善兼容性测试用例，补齐适配短板，保障

全场景、全设备的稳定使用体验。

## 七、总结

本次智能在线编程练习平台的前端开发工作，是对我前端专业技术能力、项目实战能力的一次全面检验与提升。项目开发过程中，我成功攻克了在线编辑器集成适配、多维数据可视化、前后端接口统一联调等核心技术难点，高质量完成了所有前端模块的开发、优化与迭代工作，同时清晰认识到自身在技术深度、交互设计、细节适配等方面的不足。

未来，我将针对自身短板针对性学习提升，持续沉淀本次项目的开发经验，积极配合团队完成平台后续的迭代更新与日常维护工作，持续优化前端功能与使用体验，保障平台稳定高效运行，助力项目实现降低算法学习门槛、提升用户编程核心素养的核心目标。